[7]

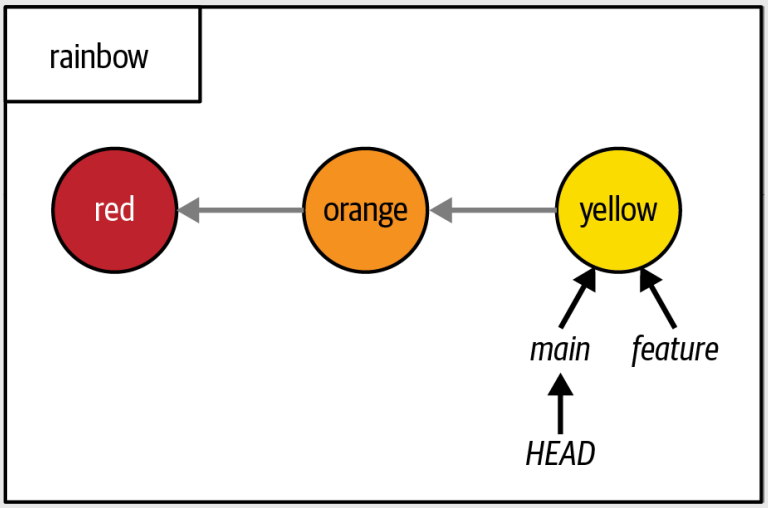
Creating and Pushing to a Remote Repository

In the last chapter, we went over choosing which hosting service you want to use, creating an account, and setting up authentication details to securely connect to remote repositories over HTTPS or SSH.
In this chapter, we will look at the different ways you can use either local or remote repositories to start working on a Git project and why remote repositories are useful. You will carry out the steps to create a remote repository for the Rainbow project, and you'll upload some data to it. Additional resources to assist you as you work through this chapter are available in the Learning Git repository (*https://github.com/gitlearningjourney/learning-git*).

State of the Local Repository

At the start of this chapter you should have three commits and two branches in your rainbow repository, and you should be on the main branch. The current state of the rainbow repository is illustrated in Visualize it 7-1.

[VISUALIZE IT 7-1]



The rainbow repository at the start of Chapter 7

The Two Ways to Start Work on a Git Project

In Chapter 2 you learned that there are two types of repositories: local repositories, which are stored on a computer, and remote repositories, which are hosted on a hosting service. You can start working on a project using Git from either a local or a remote repository. Up until now in the Rainbow project, you have taken the first approach because you started working on your project from a local repository. In Chapter 8, you will walk through a hands-on example of starting to work on a Git project from a remote repository. I'll provide a brief overview of each approach in this section.

STARTING FROM A LOCAL REPOSITORY

To start to work on a Git project from a local repository, you must first create a local repository on a computer using the git init command and make at least one commit. Next, you must create a remote repository on a hosting service. Finally, you may upload data from the local repository to the remote repository.

In Git, we use the term *push* or *pushing* to refer to the process of uploading data from a local repository to a remote repository, and the command we use to do this is git push.

[SAVE THE COMMAND]

git push

Upload data to a remote repository

This is the approach you are going to take with the Rainbow project in this chapter, and it is illustrated in Figure 7-1.

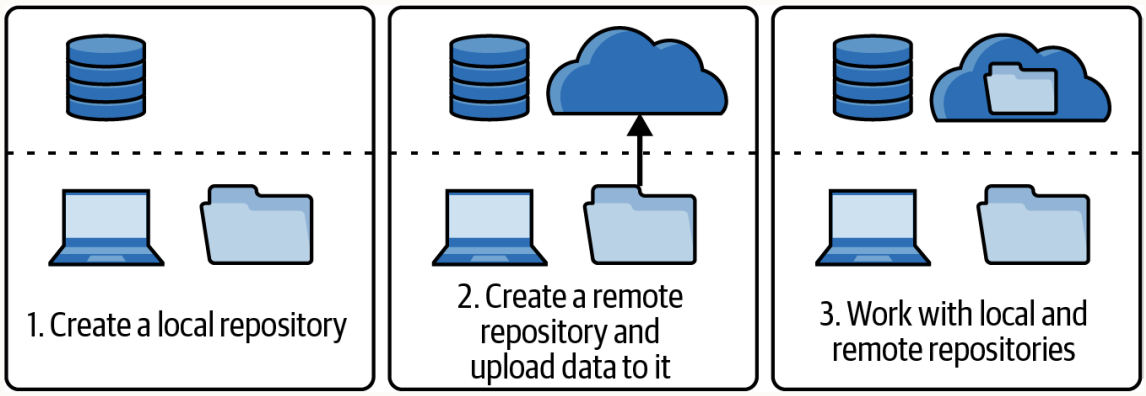


FIGURE 7-1

Starting to work on a Git project from a local repository

An example of a situation where you would start to work on a Git project from a local repository is if you have a project on your computer that you've been working on for a while that is not a repository—in other words, you are not using Git to version control it—and then you decide that you want to start version controlling it with Git. In this case, you would use the git init command to initialize the local repository on your computer, make an initial commit, and then carry out the rest of the steps to create a remote repository and upload data to it.

The other way to start working on a Git project is to start from a remote repository.

STARTING FROM A REMOTE REPOSITORY

To start to work on a Git project from a remote repository, you can either find a remote repository that you want to work on or create a new remote repository on a hosting service. Then, you *clone* (in other words, copy) the remote repository to your computer, which will create a local repository. This is the approach we will cover in Chapter 8, and it is illustrated in Figure 7-2.

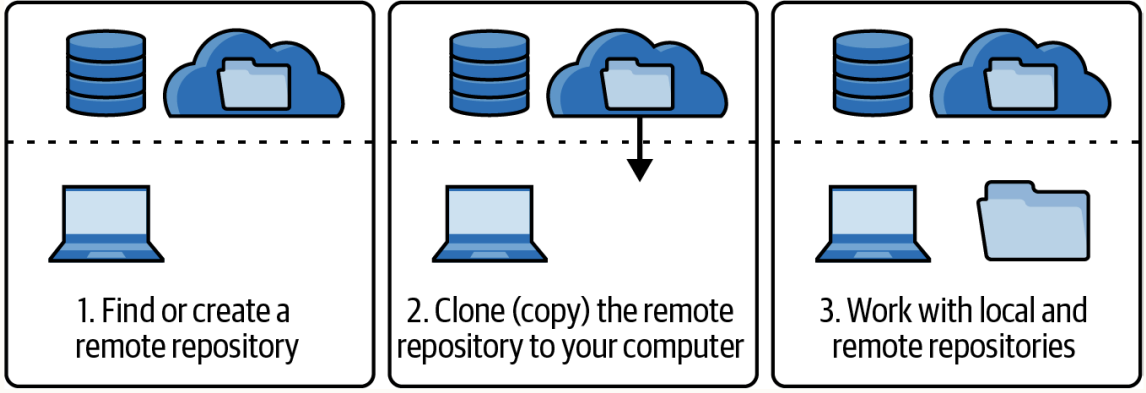


FIGURE 7-2

Starting to work on a Git project from a remote repository

An example of when you would start working on a project from a remote repository is if your friend is working on a project that has a remote repository, and they ask you to contribute to it. In this case, you would ask your friend to tell you where to find the remote repository, and then you would clone (or copy) it to your computer and start working on it.

Up to this point, with the Rainbow project you've been working exclusively with a local repository. Later in this chapter, you will create a remote repository and upload data to it. But first, I want to make a quick point about the interaction between local repositories and remote repositories.

The Interaction Between Local and Remote Repositories

Local repositories and remote repositories act separately. When it comes to working with them, it's important to understand that no interaction between them happens automatically. In other words, no updates from the local repository to the remote repository will happen automatically, and conversely, no updates from the remote repository to the local repository will happen automatically.

There is no live connection between the two. Any changes in either will be the result of you explicitly executing commands. In this and the following chapters, you will learn what these commands are.

Before you go on to create a remote repository, let's discuss why you would want to create one in the first place.

Why Do We Use Remote Repositories?

There are three main reasons why remote repositories are useful and important when working on a Git project. They allow you to:

- Easily back up your project somewhere other than your computer.
- Access a Git project from multiple computers.
- Collaborate with others on Git projects.

Let's take a look at Example Book Project 7-1 to see why you might want to create a remote repository for a Git project.

EXAMPLE BOOK PROJECT 7-1

Suppose I had only a local repository on my computer for my Book project, and no remote repository. If someone stole my laptop or the hard drive died, I would lose all the work I had done on my book. Remote repositories are a good backup mechanism.

A remote repository also will allow me to access my project from other computers. Imagine I have two laptops—one at home and one in my office that I work on during the day—and my Book project is stored on the laptop at home. If I have a remote repository for it as well, I can also access the Book project from my work laptop. Then I can either make changes to it directly on the hosting service website, or I can clone the remote repository onto my work laptop to create a local repository. An additional benefit is that I can show my progress on the book to my coworkers.

A remote repository also allows me to collaborate easily with others on my Book project. I can give access to the remote repository to anyone that wants to view my progress on it or contribute to it. Other people will then have the ability to either clone the remote repository onto their computer to make a local repository, or to simply view the contents of the remote repository directly on the hosting service website. For example, I need a remote repository to collaborate on my Book project with my coauthor and my editor.

Example Book Project 7-1 illustrates why working with remote repositories is so useful. Now that you understand this, let's go over how to create one. Suppose you've told a friend about your experience working on the Rainbow project, and they want to see what you've been doing. To make this possible, you'll need to create a remote repository. Let's do that now.

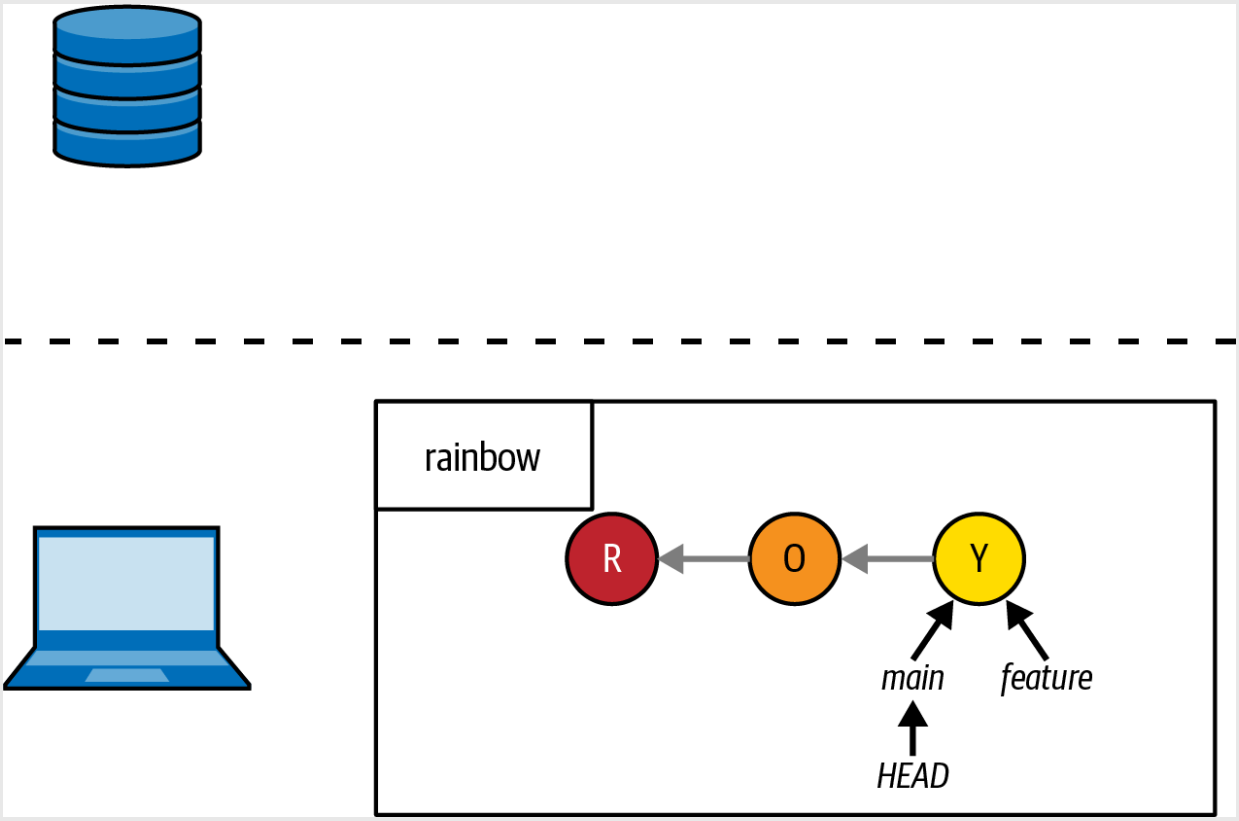
Creating a Remote Repository with Data

The Rainbow project currently consists of one local repository: your rainbow repository, shown in Visualize it 7-2.

[NOTE]

From this point forward in the book, we are going to expand the Repository Diagram to include a space at the top to represent what is happening in the remote repository, as seen in Visualize It 7-2.

[VISUALIZE IT 7-2]



The Rainbow project currently consists of one local repository, called rainbow

Your friend wants to see what you've been working on. To enable this, you need to create a remote repository and upload some data to it. There are three steps to this process:

1. Create the remote repository on the hosting service.
2. Add a connection to the remote repository in the local repository.
3. Upload (or push) data from the local repository to the remote repository.

In the following sections we'll walk through each of these, and along the way I'll introduce some new kinds of branches.

CREATING THE REMOTE REPOSITORY

When you create a remote repository on a hosting service, you will give it a *remote repository project name* and the hosting service will provide a *remote repository URL*. The remote repository URL will automatically include the remote repository project name. As you learned in Chapter 6, the hosting service will generate an SSH URL and an HTTPS URL for your repository. You should use the one for the protocol you have chosen to use.

The process of creating the remote repository is done entirely on the hosting service's website. You will need to consult your hosting service's documentation for details on the specific steps to follow. For additional resources, go to the Learning Git repository (*https://github.com/gitlearningjourney/learning-git*).

Here, I'll provide you with some general guidance. As mentioned previously, when you create your remote repository you will have to provide the project name. For the Rainbow project, this will be rainbow-remote.

[NOTE]

It is common for a remote repository and a local repository to share the same name. However, to make it easier for you to distinguish between the local and remote repositories in the Rainbow project, we'll give them distinct names.

You also will have to choose whether the remote repository will be public or private. This is a setting you can adjust on a hosting service for each repository. A *public repository* is visible to anyone on the internet. A *private repository* is visible only to the individuals given access to it. In both cases, you can contribute to the repository only if you are provided with access. Since the repository you are working on in this book is for learning purposes only, I recommend you make it private. This way you can control who is able to view it (for example, you can grant access to the friend who wants to see what you've been working on).

Some hosting services may ask if you want to include any files when you create the repository, such as a README or .gitignore file. For the purposes of this book, you should not include any files; if the option to include them is selected, make sure to deselect it. Since you will be uploading data from your local repository to the remote repository, you want your remote repository to be empty.

If the hosting service provides an option to choose a license, you can ignore this (i.e., don't select a license), as this is just a learning exercise.

Finally, some hosting services may ask you to enter a value for the default branch name. For the exercises in this book, you may either leave this value blank or enter main as the value.

Now, go to Follow Along 7-1 to create your remote repository.

[FOLLOW ALONG 7-1]

1. Log in to your account on your hosting service of choice.

2. Complete the steps to create a remote repository. For more information on this, go to the Learning Git repository (*https://github.com/gitlearningjourney/learning-git*) or consult your hosting service's documentation directly.
   When creating the repository for this exercise:

   - For the repository name, use rainbow-remote.
   - You may choose to make the repository public or private. I recommend making it private.
   - Do not include any files. For example, do not include a README file or a .gitignore file.
   - If you are asked to provide a default branch name, you may leave the field blank or set it to main.

3. Once you've finished the steps to create the remote repository, locate the remote repository URL. There will be two versions of the URL, one for HTTPS and one for SSH. In an upcoming exercise, you will have to use the URL for the protocol you set up in Chapter 6. In the examples in this book, the two remote repository URLs are:

   - HTTPS: https://github.com/gitlearningjourney/rainbow-remote.git
   - SSH: git@github.com:gitlearningjourney/rainbow-remote.git

What to notice:

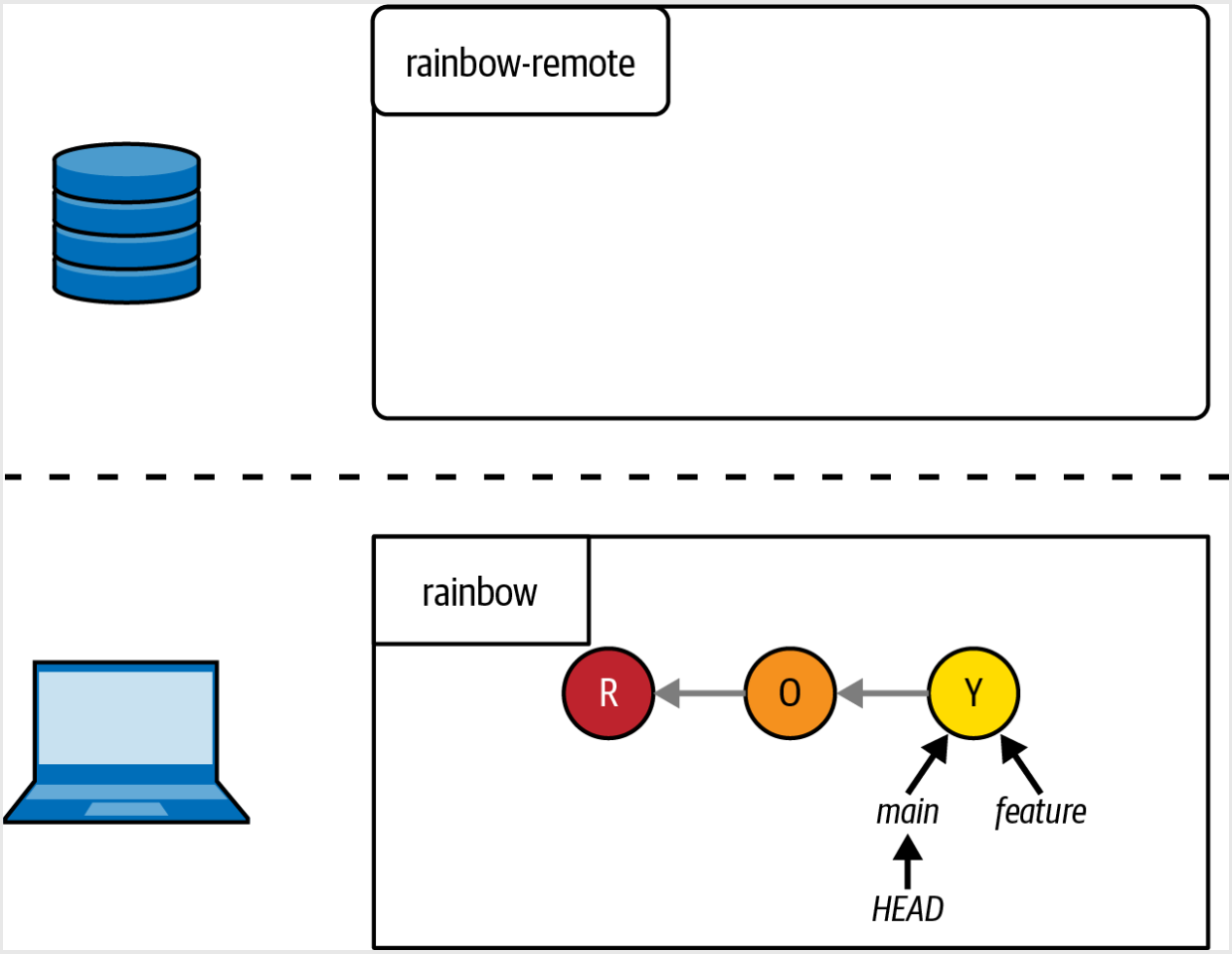
- You created a remote repository called rainbow-remote; however, there is no data in the remote repository.

The state of the Rainbow project after completing the Follow Along is illustrated in Visualize it 7-3.

[NOTE]

From this point forward, in the Repository Diagram, we use a rectangle with normal 90-degree corners to represent a local repository and a rectangle with rounded corners to represent a remote repository.

[VISUALIZE IT 7-3]



The Rainbow project after you create the empty remote repository, rainbow-remote

You've now created a remote repository on your hosting service, but as this Visualize It shows, it's currently empty. Creating a remote repository on a hosting service does not upload any data to it. To upload data, you need to first add a connection to the remote repository in your local repository. Let's take a look at how to do this.

ADDING A CONNECTION TO THE REMOTE REPOSITORY

A local repository can communicate with a remote repository when the local repository has a connection to the remote repository stored within it. This connection will have a name, which we refer to as the *remote repository shortname* or just *shortname*. A local repository can have connections to multiple remote repositories, although this isn't very common.

If a local repository was initialized locally, to set up a connection to a remote repository you must explicitly associate the remote repository URL to the remote repository shortname. To do this you use the git remote add command, passing in the shortname followed by the remote repository URL.

[SAVE THE COMMAND]

git remote add <shortname> <URL>

Add a connection to a remote repository named <shortname> at <URL>

[NOTE]

The examples in this book will use the HTTPS URL; however, you may choose to use either your SSH URL or your HTTPS URL, depending on which protocol you have chosen to use. Be sure to use the URL provided by your hosting service, which will be different from the URL used in this book's examples.

Once a connection to a remote repository is stored in a local repository, you are able to connect to the remote repository by referring to the shortname rather than the URL in the command line.

[NOTE]

When you clone a remote repository to create a local repository, Git automatically adds a connection to the remote repository with the default shortname origin. You will see this in practice in "Cloning a Remote Repository" on page 114. It is common even when a local repository is initialized locally to use origin for the shortname, as this is the default shortname for a repository when it is cloned.

This is a similar concept to how, as mentioned in "A Bit of Git History: master and main" on page 48, when we initialize a repository using the git init command with no additional options Git automatically creates a branch with the default name master. Again, there is nothing special about the terms origin or master; they are simply the current default naming conventions used by Git.

The rainbow repository was initialized locally, so you are going to have to add a connection to the remote repository in the local repository explicitly. Before you go ahead and do that, let's first go over some additional useful commands.

[NOTE]

In the official Git documentation, a connection to a remote repository stored in a local repository is simply referred to as a *remote*.

To see the list of connections to remote repositories stored in a local repository by shortname, you may use the git remote command. If you pass in the -v option (which stands for "verbose") to the git remote command, then it lists the connections to remote repositories stored in a local repository by shortname along with their URLs.

[SAVE THE COMMAND]

git remote

List the remote repository connections stored in the local repository by shortname

git remote -v

List the remote repository connections in the local repository with shortnames and URLs

Go to Follow Along 7-2 now, and associate your URL to the shortname origin in the rainbow repository. In step 4, be sure to enter the entire command on a single line.

[FOLLOW ALONG 7-2]

1. Using a filesystem window, go to rainbow > .git > config. Open the config file. You will see that at the moment there are no connections to remote repositories listed in the file. In other words, you will not see any mentions of your remote repository URL or shortname in the file.

2. Go to your hosting service and copy the remote repository URL for the protocol you have chosen (either SSH or HTTPS). You will need to use this URL in step 4 of this Follow Along.

3. rainbow $ **git remote**

4. rainbow $ **git remote add origin** https://github.com/gitlearningjourney/rainbow-remote.git

5. rainbow $ **git remote**
   origin

6. rainbow $ **git remote -v**
   origin https://github.com/gitlearningjourney/rainbow-remote.git (fetch)
   origin https://github.com/gitlearningjourney/rainbow-remote.git (push)

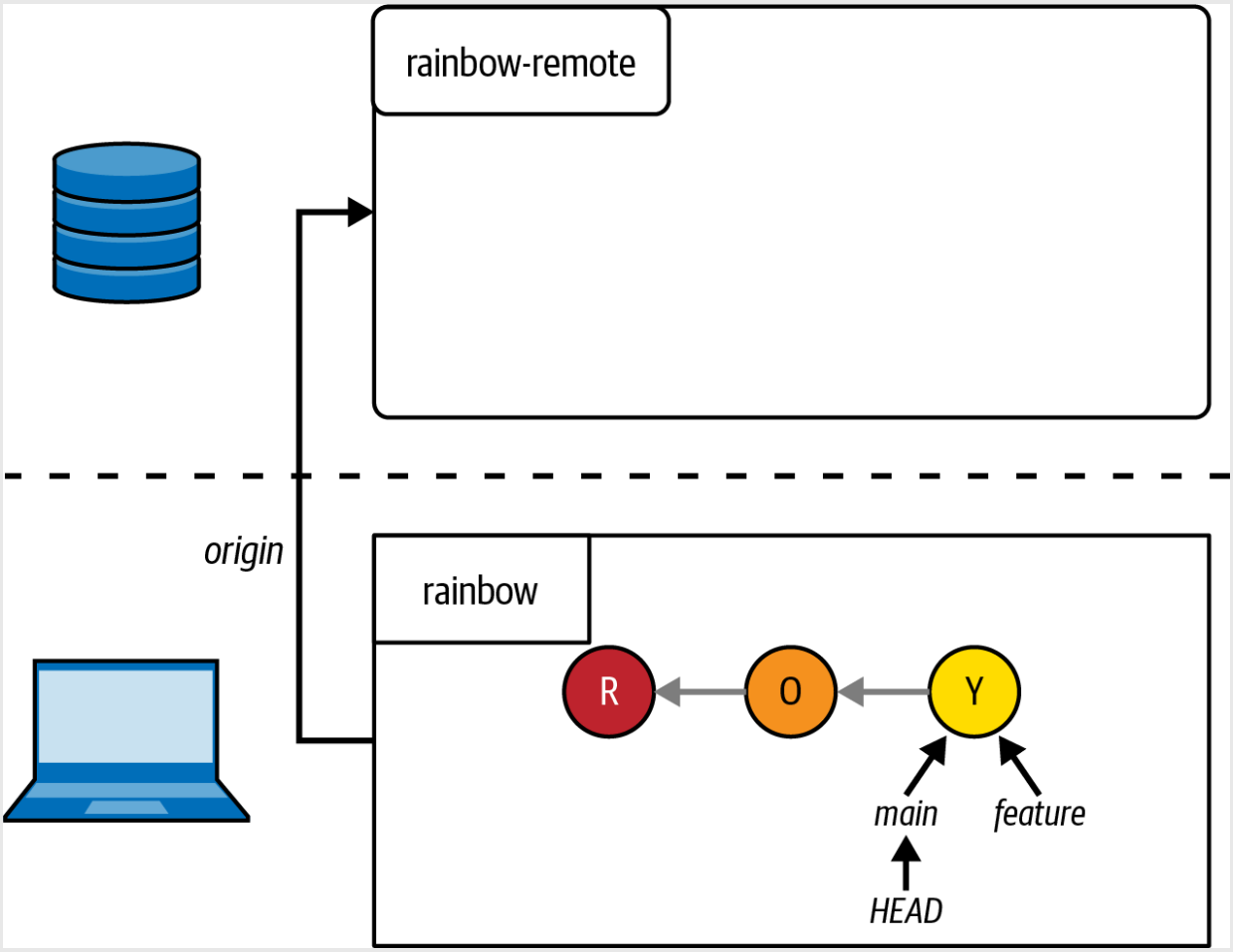
7. Close the config file and open it again in a new window. You will see that there is one connection to a remote repository listed in the file. It should look similar to:
   [remote "origin"]
   url = https://github.com/gitlearningjourney/rainbow-remote.git
   fetch = +refs/heads/*:refs/remotes/origin/*

What to notice:

- In step 3, there are no shortnames listed under the git remote output.
- In step 5, there is one shortname listed under the git remote output, which is origin.

The addition of this connection is illustrated in Visualize it 7-4.

[VISUALIZE IT 7-4]



The Rainbow project after you add a connection to the rainbow-remote repository in your rainbow repository

What to notice:

- The rainbow repository has a shortname associated with the remote repository URL, called origin.
- The rainbow-remote repository still does not have any data in it.

In Visualize it 7-4, you can see that the arrow that represents the shortname stored in the rainbow repository that relates to the rainbow-remote repository is going in only one direction. This is because the connection between a local and a remote repository goes from the local repository to the remote repository but not the other way around. In a local repository, you can find a list of all the remote repositories it has stored a connection to; however, in a remote repository you cannot find a list of local repositories that store a connection to that remote repository.

Notice as well that just because you added a connection to the remote repository in your local repository does not mean that any data from the local repository was uploaded to the remote repository. To upload data to a remote repository, you must push a branch to the remote repository. This process will upload all the commits that are part of that branch.

To help you understand what happens when you carry out this final step, I need to introduce a few other kinds of branches.

INTRODUCING REMOTE BRANCHES AND REMOTE-TRACKING BRANCHES

In Chapter 4 you learned about branches, which as you saw are movable pointers to commits. Up until now, you have worked only with local branches. When you push a local branch to a remote repository, you will create a *remote branch*. A remote branch is a branch in a remote repository.

Remote branches *do not* automatically update when you make more commits on local branches. You have to explicitly push commits from a local branch to a remote branch. Every remote branch (that a local repository knows about) also has a *remote-tracking branch*. This is a reference in a local repository to the commit a remote branch pointed at the last time any network communication happened with the remote repository. You can think of it as being like a bookmark.

You can set up a tracking relationship between a local branch and a remote branch by defining which remote branch a local branch should track. This is referred to as the *upstream branch*. There are some cases where Git will set the upstream branch automatically, but in other cases you have to set it explicitly.

When you push work from a local branch to a remote branch, Git needs to know which remote branch you want to push to. If the local branch has an upstream branch defined for it, you can use git push with no arguments, and Git will automatically push the work to that branch. However, if no upstream branch is defined for the local branch you're working on, you'll need to specify which remote branch to push to when you enter the git push command. (If you don't, you'll get an error message.)

In this chapter, we will show how to specify the remote branch you want to push to when using the git push command. In Chapter 9, you will learn how to define an upstream branch.

You're now ready to complete the third step of creating a remote repository: pushing a local branch to the remote repository.

PUSHING TO A REMOTE REPOSITORY

To push a local branch to your remote repository, you will use the git push command and pass in the shortname for the remote repository and the name of the branch that you want to push. For the rainbow repository, the shortname you will use is origin and the branch you will push is main.

[SAVE THE COMMAND]

***git push** <shortname> <branch_name>*

Upload content from <branch_name> to the <shortname> remote repository

After you execute the git push command, two things will happen:

1. A remote branch will be created in your remote repository.
2. A remote-tracking branch will be created in your local repository.

[NOTE]

To push data to a remote repository, you must be connected to the internet and have either SSH access or HTTPS access to your hosting service of choice.

In Chapter 4, you used the git branch command to list all the local branches. Now you will use the git branch command with the --all option to list all the local and remote-tracking branches in your local repository.

[SAVE THE COMMAND]

git branch --all

List local branches and remote-tracking branches

Go to Follow Along 7-3 to push your branch to the remote repository.

[FOLLOW ALONG 7-3]

1. Using a filesystem window, go to rainbow > .git > refs and look at the directories that are inside the refs directory. There should be two directories: heads and tags.

2. rainbow $ **git push origin main**
   Enumerating objects: 9, done.
   Counting objects: 100% (9/9), done.
   Delta compression using up to 4 threads
   Compressing objects: 100% (5/5), done.
   Writing objects: 100% (9/9), 747 bytes | 373.00 KiB/s, done.
   Total 9 (delta 1), reused 0 (delta 0), pack-reused 0
   remote: Resolving deltas: 100% (1/1), done.
   To github.com:gitlearningjourney/rainbow-remote.git
   * [new branch] main -> main

3. In the filesystem window, look at the directories that are inside the refs directory again. There should now be three directories: heads, tags, and remotes.

4. Go to your rainbow-remote repository on your hosting service and refresh the page. Go to the page that lists your commits. You should see your three commits there now.

5. rainbow $ **git branch --all**
   feature
   * main
   remotes/origin/main

6. rainbow $ **git log**
   commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**HEAD -> main**, origin/main, **feature**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 10:09:59 2022 +0100
   yellow
   commit 7acb333f08e12020efb5c6b563b285040c9dba93
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:42:07 2022 +0100

[FOLLOW ALONG 7-3]

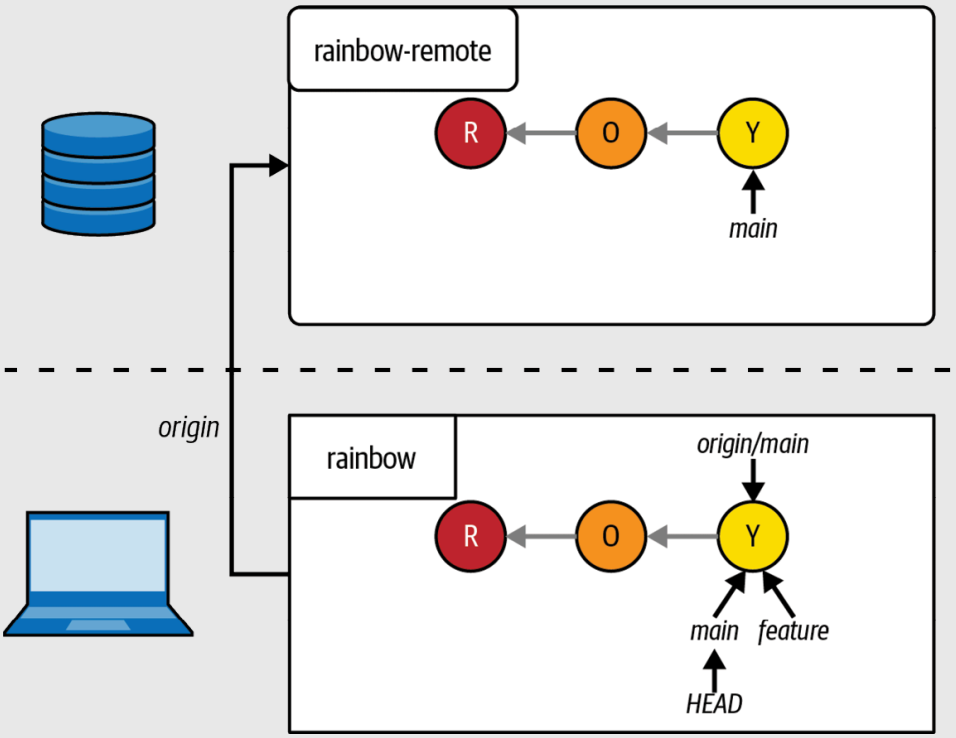
orange
commit c26d0bc371c3634ab49543686b3c8f10e9da63c5
Author: annaskoulikari <gitlearningjourney@gmail.com>
Date: Sat Feb 19 09:23:18 2022 +0100
red

What to notice:

- In step 2, the output of the git push command indicates that you have pushed your branch to the remote repository. Note that it doesn't matter if the numbers in your output are slightly different.
- In step 3, you can see that the refs directory has a new directory inside it called remotes. Inside the remotes directory is a directory called origin, and inside this directory is a file called main. This represents the new origin/main remote-tracking branch.
- In step 4, you can see that in the rainbow-remote repository, there is a remote main branch. If you go to your hosting service, you will find the three commits of the local main branch.
- There is no feature branch in the remote repository.

All of these observations are illustrated in Visualize it 7-5.

[VISUALIZE IT 7-5]



The Rainbow project after you push your local main branch to the remote repository

[NOTE]

For commands like git commit, git push, and git merge, it doesn't matter if your output has slightly different numbers than the output in this book. The same goes for other commands you'll use in later chapters in this book such as git clone, git fetch, and git pull.

Notice that when you push a specific branch to a remote repository, only the data from that branch is uploaded to the remote repository. You pushed the main branch to the remote repository, but the feature branch was not pushed to the remote repository.

At the moment, in your rainbow repository, the main and feature branches have the same commits. In other words, their development histories, which can be traced by following the commits and the parents links backward, consist of the same commits. Therefore, it is not so obvious in Visualize it 7-5 that the feature branch is not in the re-

mote repository. However, in real-world projects, your branches will often have different development histories and therefore consist of different commits. This means that if you don't push a particular branch to the remote repository, you won't have some of the commits in your remote repository.

If you also want to push your feature branch to the remote repository, you have to do so explicitly. Go to Follow Along 7-4 and do this now.

[FOLLOW ALONG 7-4]

1. rainbow $ **git switch feature**
   Switched to branch 'feature'

2. rainbow $ **git push origin feature**
   Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
   remote:
   remote: Create a pull request for 'feature' on GitHub by visiting:
   remote: https://github.com/gitlearningjourney/rainbow-remote/pull/new/feature
   remote:
   To github.com:gitlearningjourney/rainbow-remote.git
   * [new branch] feature -> feature

3. rainbow $ **git branch --all**
   * feature
   main
   remotes/origin/feature
   remotes/origin/main

4. Go to your rainbow-remote repository on your hosting service and refresh the page. Look at the list of branches.

5. rainbow $ **git log**
   commit fc8139cbf8442cdbb5e469285abaac6de919ace6 (**HEAD -> feature**, **origin/main**, **origin/feature**, **main**)
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 10:09:59 2022 +0100
   yellow
   commit 7acb333f08e12020efb5c6b563b285040c9dba93
   Author: annaskoulikari <gitlearningjourney@gmail.com>
   Date: Sat Feb 19 09:42:07 2022 +0100
   orange

[NOTE]

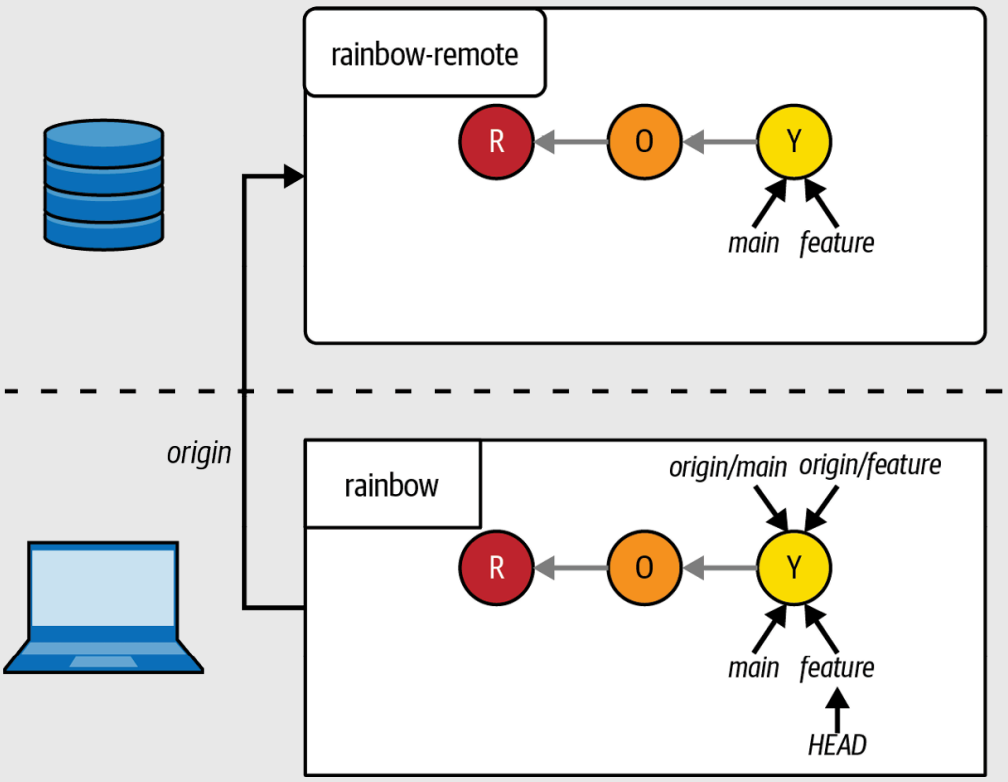
From this point forward in the book, the git log output in Follow Alongs will display only the last few commits that were made in a given repository.

What to notice:

- In step 3, you can see that in the rainbow repository there are two remote-tracking branches, origin/main and origin/feature.
- In step 4, you can see that in the rainbow-remote repository there are two remote branches, main and feature.

This is illustrated in Visualize it 7-6.

[VISUALIZE IT 7-6]



The Rainbow project after you push the local feature branch to the remote repository

You have now created a remote repository with data in it. In Chapter 6, I mentioned that there are two ways to make changes to a remote repository:

1. By logging in to the hosting service via its website and making changes there directly.
2. By making changes in your local repository and uploading those changes to the remote repository on the hosting service.

In this chapter we covered an example of the second situation, but let's talk briefly about the first one.

Working on a Remote Repository Directly on a Hosting Service

Up until now, you have learned how to use your local repository to make commits (Chapter 3), work with branches (Chapter 4), and carry out a basic merge (Chapter 5).

However, it is also possible to use the UI of a hosting service's website to carry out a lot of actions (and many more) that are similar to these. For example, on a hosting service you can make commits directly on a remote repository, you can create remote branches, and, as you will see in Chapter 12, you can merge branches through a feature called a pull request.

Because this book focuses on teaching how to use Git in the command line, we won't cover how to carry out all the corresponding actions in a hosting service. However, all of these features will be comprehensively documented in your hosting service's documentation. These features may be helpful for collaborators you work with in the future that don't know how to use Git.

Now that you have learned the basics of working with local and remote repositories and you have created a remote repository for the Rainbow project and pushed some branches to it, the next step in your learning journey is to start collaborating on the project.

Summary

This chapter introduced what remote repositories are, why we use them, and the three steps to create a remote repository with data in it when you're starting from a local repository. You set up a remote repository on your hosting service, added a connection to the remote repository in your local repository, and pushed data from the local reposi-

tory to the remote repository. Along the way, you learned about the concepts of remote branches, remote-tracking branches, and upstream branches.

Next, in Chapter 8, your friend will start helping you work on your project by cloning the remote repository.

Previous chapter

‹ Chapter 6. Hosting Services and Authentication

Next chapter

Chapter 8. Cloning and Fetching ›